

RESEARCH ARTICLE

SAC-RAG: Semantic Adaptive Context Compression for Retrieval-Augmented Generation

DEYU ZHANG¹, HONGQIANG YU¹, JINZE HUO¹, AND MO SU

School of Information Science and Engineering, Shenyang Ligong University, Shenyang 110159, China

Corresponding author: Jinze Huo (huojinze@sylu.edu.cn)

This work was supported by Liaoning Provincial Science and Technology Plan Joint Program under Grant 2025JH2/101800311.

ABSTRACT Retrieval-Augmented Generation (RAG) effectively mitigates the knowledge cutoff and hallucination issues of large language models by incorporating external knowledge bases. However, standard RAG applies a uniform retrieval strategy to all question types and feeds redundant retrieval results directly into the generator, leading to high token consumption and potentially degraded answer quality. This paper proposes SAC-RAG (Semantic Adaptive Compress RAG), a unified framework that integrates question-type-adaptive retrieval with context compression. SAC-RAG first classifies queries into three types—factual, definitional, and reasoning—via an LLM classifier, applies a differentiated retrieval strategy for each type, then uses a unified LLM context compression module to extract evidence snippets, and finally generates answers from the compressed and refined context. To independently quantify the contribution of each component, we design four progressive ablation experiments on three multi-hop QA datasets—HotpotQA, 2WikiMultiHopQA, and MuSiQue—and evaluate using four metrics: Exact Match, F1, Recall@10, and token consumption. Experimental results show that SAC-RAG reduces token consumption by 38%–58% at the cost of only a 1–2 percentage point EM drop, with EM actually improving after compression for reasoning-type questions, achieving the optimal quality–efficiency trade-off in terms of token consumption, with a moderate increase in per-query latency due to additional LLM calls. SAC-RAG offers a modular and interpretable optimization solution for the efficient deployment of RAG in real-world settings.

INDEX TERMS Context compression, multi-hop question answering, question-type-adaptive retrieval, retrieval-augmented generation, semantic classification.

I. INTRODUCTION

Large Language Models (LLMs) perform well across a wide range of natural language processing tasks, yet they suffer from two inherent limitations: knowledge cutoff and factual hallucination. Retrieval-Augmented Generation (RAG) enhances LLM responses by retrieving relevant documents from external knowledge bases and has become the dominant approach for addressing these issues. However, standard RAG systems have two critical blind spots. First, they apply a uniform retrieval strategy to all questions, ignoring the fact that different question types require

differentiated retrieval approaches. Second, they feed all retrieved document chunks directly into the generator, introducing substantial noise and redundancy [1], which incurs high token consumption and may degrade answer quality. Recent studies further reveal that relevant evidence can still be overlooked within long retrieved contexts, leading to a “lost-in-retrieval” phenomenon [2] that calls for more targeted context handling.

Existing work typically optimizes retrieval or compression in isolation. On the retrieval side, Adaptive-RAG [3] dynamically selects a retrieval strategy (no retrieval / single-step retrieval / multi-step retrieval) based on question complexity, but it does not incorporate a context compression mechanism. Knowledge-guided iterative retrieval methods

The associate editor coordinating the review of this manuscript and approving it for publication was Alba Amato¹.

such as KiRAG [4] further enhance evidence discovery through multi-step retrieval and knowledge triples, yet they focus on retrieval–reasoning synergy without addressing compression. On the compression side, methods such as LLMLingua [5] and LongLLMLingua compress retrieved context on top of standard retrieval, effectively reducing token consumption; however, they operate on a fixed retrieval pipeline without considering question-type differences in the retrieval stage. To the best of our knowledge, no prior work unifies question-type-adaptive retrieval and context compression within a single framework.

To address these limitations, this paper proposes SAC-RAG (Semantic Adaptive Compress RAG), a unified framework that integrates question-type-adaptive retrieval with context compression. SAC-RAG first classifies each user query into one of three semantic types via an LLM classifier: Fact, Definition, and Reasoning. SAC-RAG employs a differentiated retrieval strategy for each type. Fact-type questions use hybrid retrieval combining BM25 sparse retrieval and Milvus dense vector retrieval with BGE cross-encoder reranking. Definition-type questions adopt pure semantic vector retrieval. Reasoning-type questions are decomposed into multiple sub-questions by an LLM, with results merged after parallel retrieval. Once retrieval is complete, results of all types pass through a unified LLM context compression module to extract evidence snippets, which are then fed into the generator to produce the final answer.

To systematically validate the independent contribution of each component, we design four progressive ablation settings on three multi-hop question answering datasets—HotpotQA [6], 2WikiMultiHopQA [7], and MuSiQue [8]—namely Naive RAG (baseline), Compress RAG (compression only), Adaptive RAG (adaptive retrieval only), and SAC-RAG (the full framework). Evaluation covers four metrics: Exact Match (EM), F1, Recall@10, and average token consumption.

Experimental results show that: (1) adaptive retrieval is the primary driver of answer quality improvement, boosting EM from 0.560 to 0.611 (+9.1%) and Recall@10 from 0.787 to 0.880 (+11.8%) on HotpotQA; (2) context compression reduces token consumption by 55–58% with EM decreasing by only 1–2 percentage points; (3) SAC-RAG achieves the optimal quality–efficiency trade-off; on Reasoning-type questions, EM improves after compression (HotpotQA: 0.455→0.577). Unlike previous studies that optimize either retrieval or compression independently, SAC-RAG coordinates semantic-aware retrieval and context compression within a unified framework. The main contributions of this paper are as follows:

- A question-semantic-type-based adaptive retrieval framework with three differentiated retrieval strategies;
- A unified architecture, SAC-RAG, that integrates adaptive retrieval and context compression;
- Systematic ablation experiments that independently quantify the contribution of each component, providing clear guidance for RAG system optimization.

II. RELATED WORK

A. EVOLUTION OF RETRIEVAL-AUGMENTED GENERATION

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. [9] in 2020, with the core idea of retrieving relevant documents from external knowledge bases as additional context before an LLM generates an answer, thereby addressing knowledge cutoff and hallucination. Early RAG systems relied on dense vector retrieval [10], mapping queries and documents into a shared vector space for nearest neighbor search. Gao et al. [11] categorized the evolution of RAG into three stages: Naive RAG uses a fixed retrieve-then-generate pipeline; Advanced RAG introduces pre- and post-retrieval optimizations such as query rewriting, reranking, and compression; Modular RAG decomposes the system into independently replaceable functional modules, supporting iterative retrieval and adaptive scheduling. A recent systematic review [12] further confirms this taxonomy through a PRISMA-compliant analysis of 128 highly cited RAG studies, identifying modular and policy-driven architectures as the prevailing trend. Within this framework, training paradigms such as Atlas [13], context-prefix schemes such as In-Context RALM [14], and standardized benchmarks such as BEIR [15] emerged in succession. As retrieved documents grow longer, however, LLMs become less effective at using key information in extended contexts [1]; redundant content not only raises computational cost but also introduces noise that can degrade answer quality. These issues are particularly acute in multi-hop reasoning, where retrieval spans multiple documents of uneven relevance, demanding both precision in retrieval strategies and efficiency in context utilization—as well as coordination between the two. Existing work addressing these bottlenecks falls into three main directions: adaptive retrieval, context compression, and joint optimization of retrieval and reasoning.

B. ADAPTIVE RETRIEVAL METHODS

Adaptive retrieval adjusts retrieval strategies based on query characteristics to obtain more precise context. For strategy selection, Adaptive-RAG [3] routes questions among no-retrieval, single-step, and multi-step retrieval paths based on question complexity; Self-RAG [16] uses a self-reflection mechanism to trigger retrieval on demand during generation; FLARE [17] actively supplements retrieval based on token confidence during decoding. For retrieval quality, Vendi-RAG [18] introduces a diversity metric to balance coverage and relevance of retrieved documents; REPLUG [19] improves reliability through multi-retriever ensembling. On the input side, Ma et al. [20] proposed the Rewrite-Retrieve-Read framework, training a compact rewriting model to optimize queries before retrieval. More recently, LLM-based generation-augmented rewriting methods such as GaQR [21] further leverage chain-of-thought reasoning to expand queries, achieving substantial retrieval recall improvements. A parallel line of work explores query routing at the model level. RouteLLM [22] learns query-aware routing policies that balance response quality

and inference cost by dynamically selecting among heterogeneous LLMs. RAGRouter [23] further incorporates retrieved-document representations into routing decisions for selecting the most suitable RAG model per query. However, these routing methods primarily address model selection or query-complexity-based dispatch, whereas SAC-RAG adapts *retrieval strategies* within a unified pipeline according to semantic question types—a distinction that yields finer-grained control over the retrieval process.

A shared limitation of these methods is that their adaptation axis centers on question complexity or token confidence, without designing differentiated retrieval mechanisms based on question semantic type (factual, definitional, reasoning). As a result, retrieval strategies lack a structured understanding of the nature of the question.

C. CONTEXT COMPRESSION METHODS

Context compression aims to reduce redundancy in retrieval results, lowering token consumption during generation. LLMingua [5] introduced selective compression of retrieval results using a small language model; LongLLMLingua [24] extended this to long-context settings. In terms of compression strategy, RECOMP [25] formalized compression as a summarization task; BRIEF [26] introduced a query-aware mechanism that doubles the compression rate while maintaining answer quality; LLMingua-2 [27] proposed a task-agnostic data distillation approach. SARA [28] further combines fine-grained textual spans with compact semantic compression vectors to improve context efficiency under strict token budgets. The growing focus on token efficiency has also motivated agentic frameworks such as TeaRAG [29], which jointly compresses retrieved content and reasoning chains to achieve significant token reduction while preserving accuracy. A common limitation of these methods is that compression operates on top of a fixed retrieval strategy: the retrieval stage is unaware of downstream compression needs, and the compression stage does not exploit type information from the retrieval stage. The two components remain independent, lacking co-design.

D. JOINT RETRIEVAL AND REASONING

In multi-hop QA, retrieval and reasoning are not independent stages—reasoning requires multiple retrieval steps to supply intermediate evidence, and retrieval strategies shift dynamically with each reasoning step. Chain-of-Thought (CoT) prompting [30] established the approach of improving reasoning through intermediate steps; ReAct [31] interleaves reasoning and action, deciding after each reasoning step whether new retrieval is needed; IRCot [32] embeds retrieval within each step of the CoT reasoning chain. For task decomposition, DecomP [33] and DSP [34] adopt a decompose-retrieve-merge strategy; ChainRAG [2] addresses the information loss problem in sub-question decomposition through progressive retrieval-and-rewriting over a sentence graph. KiRAG [4] further incorporates knowledge triples into iterative retrieval, bridging information gaps

across reasoning steps. These works show the value of jointly optimizing retrieval and reasoning. However, they focus on retrieval–reasoning synergy, while the joint optimization of retrieval and compression remains an open problem: no existing method unifies adaptive retrieval and context compression within a single framework.

E. RESEARCH GAP AND MOTIVATION

The analysis of the three directions above reveals a shared limitation: the components of a RAG system—retrieval, compression, and reasoning—are still optimized as independent modules. Adaptive retrieval addresses how to retrieve, but does not consider how the retrieval results will be compressed; context compression addresses how to condense, but does not exploit type information from the retrieval stage to guide compression strategy; joint retrieval and reasoning demonstrates the value of co-design, but this principle has yet to be extended to retrieval and compression. The core motivation of SAC-RAG is to fill this gap: to build a unified framework in which question-type-adaptive retrieval and context compression work together. In SAC-RAG, the classifier’s output determines the retrieval strategy; the resulting type-specific retrieval outputs naturally differ in document count, redundancy, and information density, which shapes the input to the downstream compression module. Although compression itself uses a unified prompt and logic across all question types, the diversity of its inputs produces type-adaptive compression effects. This indirect coordination through the data flow distinguishes SAC-RAG from prior modular designs where retrieval and compression operate as independently configurable modules. Retrieval strategies are differentiated by semantic type, compression processes diverse retrieval outputs with a consistent logic, and the two components achieve positive complementarity in the quality–efficiency trade-off.

III. METHOD

A. SAC-RAG SYSTEM OVERVIEW

The SAC-RAG pipeline, as shown in Figure 1, consists of five core stages: question classification, type-specific retrieval, context compression, answer generation, and evaluation. Given a user query q , the system first determines its semantic type t via a question classifier, then selects the corresponding retrieval strategy to retrieve a document set from the knowledge base $\mathcal{D}$, after which a unified context compression module extracts refined evidence snippets, and the system finally generates the answer $\hat{a}$.

B. QUESTION CLASSIFIER

The question classifier uses an LLM with a structured prompt and few-shot examples to categorize queries into three types. Formally, given a query q , the classifier outputs its semantic type label:

$$c = \text{Classifier}(q), \quad c \in \{\text{Fact}, \text{Definition}, \text{Reasoning}\} \quad (1)$$

The three question types are defined as follows:

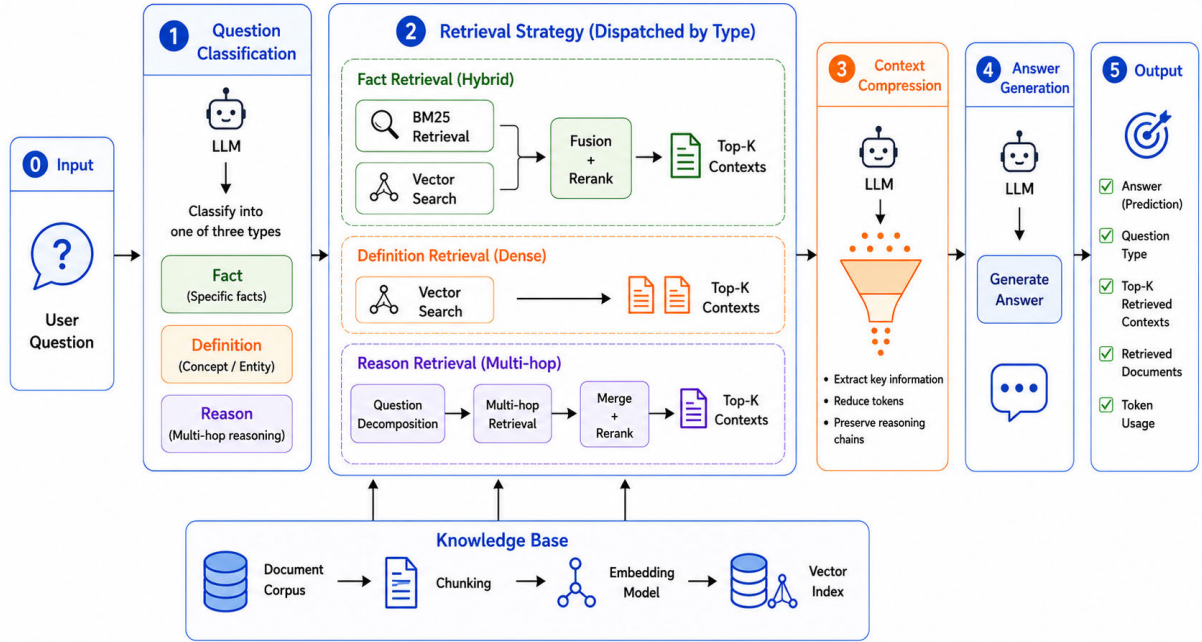


FIGURE 1. SAC-RAG consists of five pipeline stages: question classification, type-specific retrieval, context compression, answer generation, and evaluation. A query is first classified by semantic type (Fact / Definition / Reasoning), then routed to the corresponding retrieval strategy—Fact through hybrid retrieval with reranking, Definition through pure semantic retrieval, and Reasoning through sub-question decomposition with parallel retrieval—after which all retrieved results pass through a unified LLM compression module to extract evidence, and the generator produces the final answer from the compressed context. Dashed boxes indicate modules that can be independently toggled in the ablation study.

- Fact: queries about specific facts, values, or entity attributes, with typical question words including When, Where, How many, Which.
- Definition: queries about the definition of concepts or entities, with typical question words including What is, Who is, Define.
- Reasoning: queries requiring multi-step reasoning and causal analysis, with typical question words including Why, How, What caused.

C. TYPE-SPECIFIC RETRIEVAL STRATEGIES

1) FACT: HYBRID RETRIEVAL + RERANKING

Fact-type questions involve specific entities and keywords. Relying solely on dense vector retrieval may miss exact term matches due to semantic generalization. SAC-RAG adopts a hybrid retrieval strategy, where the combined score for a candidate document d is defined as follows:

$$S_{\text{Fact}}(q, d) = \lambda \cdot \text{BM25}(q, d) + (1 - \lambda) \cdot \cos(\mathbf{e}_q, \mathbf{e}_d) \quad (2)$$

where $\mathbf{e}_q$ and $\mathbf{e}_d$ are dense vectors of the query and document encoded by text-embedding-3-small, and the BM25 term matching score is computed as:

$$\text{BM25}(q, d) = \sum_{w \in q} \text{IDF}(w) \cdot \frac{f(w, d)(k_1 + 1)}{f(w, d) + k_1(1 - b + b \frac{|d|}{\text{avgdl}})} \quad (3)$$

where $f(w, d)$ is the term frequency of w in document d , $|d|$ is the document length, avgdl is the average document length,

and k_1 and b are BM25 hyperparameters (default $k_1 = 1.5$, $b = 0.75$). The specific procedure is:

- 1) BM25 sparse retrieval: based on spaCy tokenization and rank_bm25.BM25Okapi, retrieving top-10 by term matching.
- 2) Dense vector retrieval: ANN retrieval via Milvus using Cosine similarity, retrieving top-10.
- 3) Merge and deduplication: the two result sets are merged and deduplicated by doc_id.
- 4) BGE reranking: using the bge-reranker-base cross-encoder to score each candidate and selecting the top-10.

2) DEFINITION: PURE SEMANTIC RETRIEVAL

Definition-type questions focus on the meaning of a single concept, where semantic vector similarity is sufficient to cover the relevant content. For definition-type questions, SAC-RAG uses only Milvus dense vector retrieval (Cosine similarity), taking the top-8 documents directly into the subsequent compression and generation stages, avoiding unnecessary hybrid retrieval overhead.

3) REASONING: SUB-QUESTION DECOMPOSITION + PARALLEL RETRIEVAL

Reasoning-type questions require support from multiple knowledge points, and a single retrieval query may fail to cover all reasoning paths. SAC-RAG adopts

a decompose-retrieve-merge strategy:

$$\{q_1, \dots, q_k\} = \text{Decompose}(q),$$

$$\mathcal{R}_{\text{Reasoning}} = \text{Rerank}\left(\bigcup_{i=1}^k \text{Retrieve}(q_i)\right) \quad (4)$$

The specific steps are:

- 1) Sub-question decomposition: use an LLM to decompose the original query q into a set of independent sub-questions $\{q_1, \dots, q_k\}$.
- 2) Parallel retrieval: each sub-question q_i undergoes independent Milvus vector retrieval, each taking top-10.
- 3) Result merging and reranking: after merging and deduplication, results are reranked by the BGE cross-encoder, taking the top-10.

The retrieval strategy mapping is manually designed based on the distinct retrieval characteristics of different semantic question types. Compared with learning-based routing approaches, this predefined mapping avoids additional routing-model training and inference overhead while remaining interpretable and lightweight. The effectiveness of this design is validated by the consistent gains of Adaptive RAG over Naïve RAG in the ablation study (H2, see Section V-C2).

D. CONTEXT COMPRESSION

The top- k documents produced during the retrieval stage are fed into a unified LLM compression module, which extracts evidence snippets relevant to the question through a custom prompt:

$$C_t = \text{Compress}(q, \mathcal{R}_t),$$

$$t \in \{\text{Fact, Definition, Reasoning}\} \quad (5)$$

The compression uses a unified prompt and compression logic for all question types, but the input retrieved document set $\mathcal{R}_t$ varies by question type (differing in k value and retrieval strategy).

E. ANSWER GENERATION

The compressed context C_t is merged with the original query q and fed into the LLM to generate the final answer $\hat{a}$. All experimental groups use the same LLM configuration to ensure fair comparison.

IV. EXPERIMENTAL SETUP

A. DATASETS AND EVALUATION METRICS

Experiments are conducted on three English multi-hop open-domain question answering datasets. Dataset statistics and evaluation metrics are shown in Table 1. The four evaluation metrics follow standard practices in the retrieval-augmented generation literature: EM and F1 follow the answer quality evaluation approach established by DPR [10], Recall@10 assesses retrieval quality, and average token consumption (Avg Tokens) follows the efficiency metric introduced by LLMingua [5].

All answer quality metrics are computed after applying `normalize_answer()` normalization (Unicode NFKD, lowercasing, punctuation and article removal, whitespace collapsing). The formal definitions of Recall@ K and F1 are as follows:

$$\text{Recall@}K = \frac{|\mathcal{D}_{\text{ret}}^K \cap \mathcal{D}_{\text{GT}}|}{|\mathcal{D}_{\text{GT}}|} \quad (6)$$

$$F_1 = \frac{2 \cdot |\text{tok}(\hat{a}) \cap \text{tok}(a)|}{|\text{tok}(\hat{a})| + |\text{tok}(a)|} \quad (7)$$

where $\mathcal{D}_{\text{ret}}^K$ denotes the set of top- K retrieved documents, $\mathcal{D}_{\text{GT}}$ denotes the set of ground-truth supporting documents, $\hat{a}$ and a denote the predicted and ground-truth answers respectively, and $\text{tok}(\cdot)$ denotes the normalized token set.

B. EXPERIMENTAL ENVIRONMENT

The vector database uses Milvus (Collection `en_doc_chunk`, 1536 dimensions, Cosine similarity), and metadata storage uses MongoDB. BM25 sparse retrieval is based on `rank_bm25.BM25Okapi` with `spaCy` tokenization. The reranker uses BGE `bge-reranker-base` [35]. For each dataset, 1,000 questions are independently run under 4 RAG configurations, with temperature set to 0 to ensure deterministic outputs.

V. EXPERIMENTAL RESULTS AND ANALYSIS

A. OVERALL PERFORMANCE

To position SAC-RAG within the RAG research landscape, we first conduct a horizontal comparison with recent representative RAG methods, including standard retrieval-augmented generation (Vanilla RAG), iterative retrieval augmentation (IterDRAG [36]), search-augmented retrieval (ReSearch [36]), and Monte Carlo tree search-based autonomous reasoning (AirRAG [36]). These methods use different backbone LLMs and implementation settings; thus this comparison aims to evaluate overall system-level effectiveness rather than serve as a strictly controlled variable experiment.¹

As shown in Table 2, two clear patterns emerge. The baselines use different backbone LLMs (Qwen2.5-7B) and implementation settings than SAC-RAG (gpt-5-mini); therefore, this comparison evaluates overall system-level effectiveness rather than isolating framework-specific contributions. The controlled variable ablation study (Table 5) provides the internally consistent comparison. First, SAC-RAG achieves F1 scores comparable to the best-performing methods on HotpotQA and 2Wiki (74.0 vs. 74.9; 66.4 vs. 65.7), and leads significantly on MuSiQue (40.3 vs. 30.3), suggesting that question-type-adaptive retrieval offers greater advantages when retrieval difficulty is high. Second, SAC-RAG is the only method that simultaneously reports token consumption and Recall@10—average tokens of only 1,087–1,918 (vs. 2,407–3,111 for Adaptive RAG, see Table 5), with

¹N/R = Not Reported. Prior work generally does not report token consumption or Recall@10.

TABLE 1. Dataset statistics and evaluation metrics.

	HotpotQA [6]	2WikiMultiHopQA [7]	MuSiQue [8]
Questions	1,000	1,000	1,000
Type (Fact/Definition/Reasoning)	958/20/22	796/5/199	732/28/240
Hops	1–2	2	2–4
Evaluation Metrics			
EM	Normalized exact match between prediction and ground-truth after Unicode NFKD normalization, lowercasing, punctuation and article removal. [10]		
F1	Token-level harmonic mean of precision and recall. [10]		
Recall@K	$ \mathcal{D}_{\text{ret}}^K \cap \mathcal{D}_{\text{GT}} / \mathcal{D}_{\text{GT}} $, proportion of ground-truth supporting documents retrieved in the top- K results		
Avg Tokens	Mean token consumption (generation only) per query. [5]		

TABLE 2. System-level comparison with existing RAG methods.

Method	HotpotQA		MuSiQue		2Wiki		Tokens	R@10
	F1	Acc	F1	Acc	F1	Acc		
Vanilla RAG [†]	61.3	56.9	13.5	8.3	45.9	42.8	N/R	N/R
IterDRAG [†]	65.3	60.7	18.3	13.0	51.8	47.0	N/R	N/R
ReSearch [†]	70.3	68.1	30.9	27.4	62.5	61.0	N/R	N/R
AirRAG [†]	74.9	70.0	30.3	23.6	65.7	63.2	N/R	N/R
SAC-RAG[‡]	74.0	65.2	40.3	30.8	66.4	65.7	1.1k/1.9k/1.3k	.88/.48/.70

Recall@10 reaching 0.704–0.881. Prior methods generally lack both metrics, making the retrieval quality and operational efficiency of RAG systems difficult to evaluate reproducibly.

The above system-level comparison reflects SAC-RAG’s competitiveness at a macro level, but the independent contributions of the classifier, adaptive retrieval, and context compression modules remain to be quantified. The following ablation experiments and bar chart analyses on three datasets address this.

On HotpotQA, Adaptive RAG achieves the highest answer quality—EM 0.611, F1 0.751, representing improvements of 9.1% and 7.8% over Naïve RAG, respectively. SAC-RAG achieves an EM of 0.595 (a 2.6% decrease), but its average token consumption is only 1,087, a 54.8% reduction compared to Adaptive RAG’s 2,407. In terms of Recall@10, SAC-RAG reaches 0.881, an 11.8% improvement over Naïve RAG (0.787).

On 2Wiki, Adaptive RAG EM is 0.595 and SAC-RAG is 0.590 (a 0.8% decrease), while token consumption drops from 3,002 to 1,264 (−57.9%). Recall@10 improves from 0.684 to 0.704 (+3.7%).

MuSiQue Cross-Dataset Generalization. To verify the generalization capability of SAC-RAG on higher-difficulty datasets, we conduct the same experiments on MuSiQue (a 2–4 hop multi-step reasoning dataset), with results shown in Figure 3. The overall retrieval difficulty of MuSiQue is significantly higher than that of HotpotQA and 2Wiki (Recall@10 only 0.42–0.48), but the performance patterns of each strategy are highly consistent with the two aforementioned datasets: Adaptive RAG achieves the best answer

TABLE 3. Semantic classification validation against manually annotated labels.

Metric	Value
Evaluation Samples	300
Accuracy	92.0%
Macro Precision	0.748
Macro Recall	0.726
Macro F1	0.721

quality (EM 0.284, F1 0.422), SAC-RAG reduces tokens from 3,111 to 1,918 (−38.3%) at the cost of only a 1.3pp EM drop (0.284→0.271), and Recall@10 remains exactly level with Adaptive RAG (0.479). This result confirms that the SAC-RAG framework maintains a consistent quality-efficiency trade-off across datasets with different retrieval difficulties.

B. SEMANTIC CLASSIFICATION VALIDATION

To independently evaluate the semantic classification module, 300 questions were randomly sampled from the three benchmark datasets (100 per dataset) while preserving their original data distribution. Each sampled question was manually annotated according to the semantic type definitions described in Section III-B, and the resulting annotations were used as the reference labels. The semantic classifications generated by GPT-5-mini were then compared against these manually verified labels. The results are reported in Table 3.

The sampled questions preserve the original semantic distribution of the benchmark datasets. The observed errors

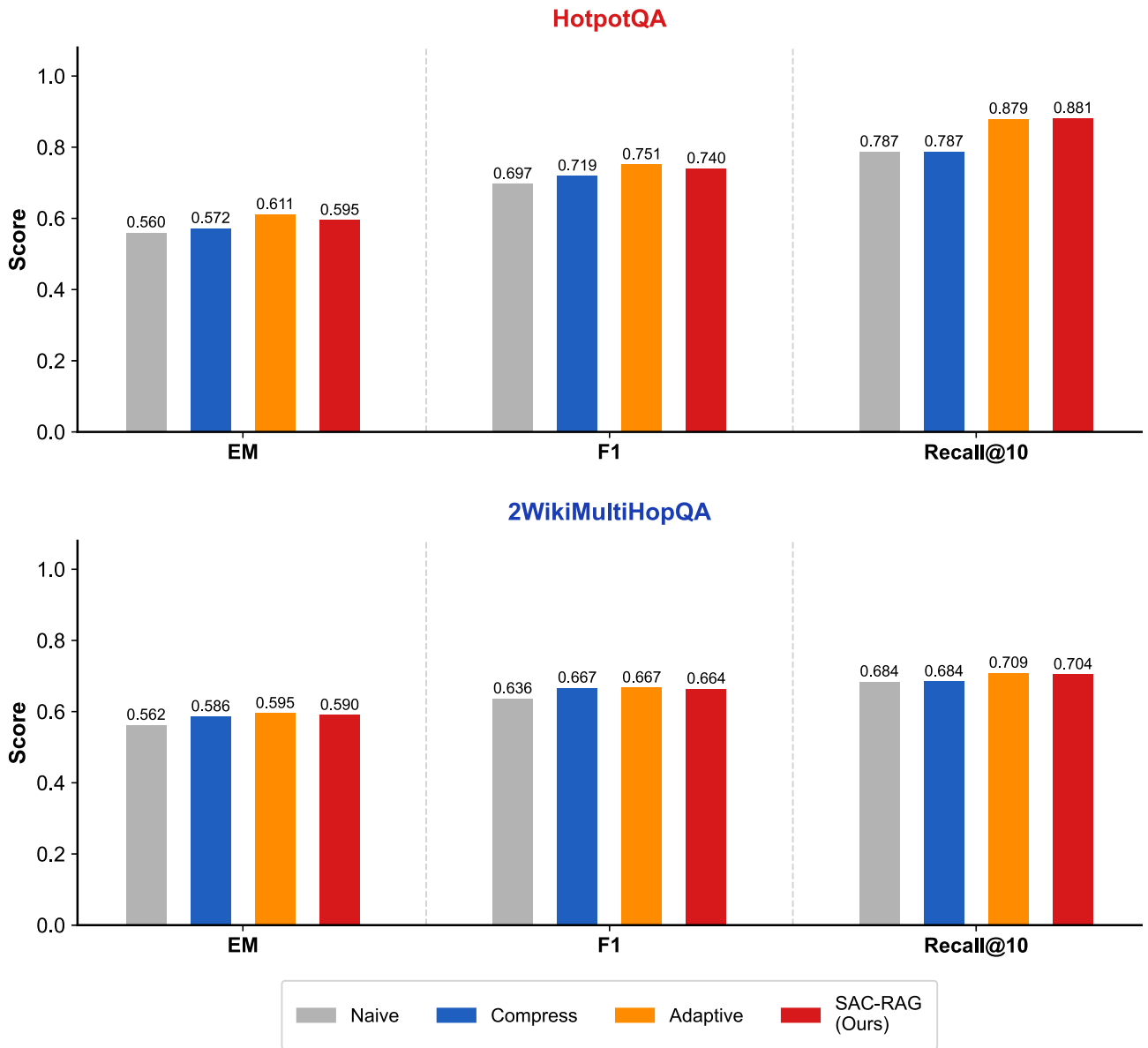


FIGURE 2. Results on HotpotQA and 2WikiMultiHopQA comparing four RAG configurations across answer quality (EM, F1) and retrieval recall (Recall@10). Each dataset evaluates 1,000 questions with temperature set to 0.

are predominantly Reasoning questions misclassified as Factual (21 of 24, 87.5%). This conservative bias avoids routing simple queries through the expensive decomposition pipeline, which would incur unnecessary latency without quality gain. The 92.0% overall accuracy indicates that semantic type prediction is sufficiently reliable to support the downstream type-specific retrieval strategy.

C. ABLATION STUDY: DESIGN AND VERIFICATION

1) EXPERIMENTAL DESIGN

To independently quantify the contribution of each component, we design four progressive ablation study groups. The configuration and retrieval parameters of each group are shown in Table 4.

Using Naïve RAG as the baseline, the remaining three groups incrementally build upon it: Compress RAG adds only the context compression module on top of Naïve RAG’s retrieval results, to independently quantify the contribution of compression; Adaptive RAG adds only the question classifier and type-specific retrieval strategies, to independently quantify the contribution of adaptive retrieval; SAC-RAG, as the full framework, enables both modules simultaneously, to verify their synergistic effect. The four groups collectively test three core hypotheses:

- **H1:** Context compression can effectively reduce token consumption while maintaining or improving answer quality (Compress vs Naïve).

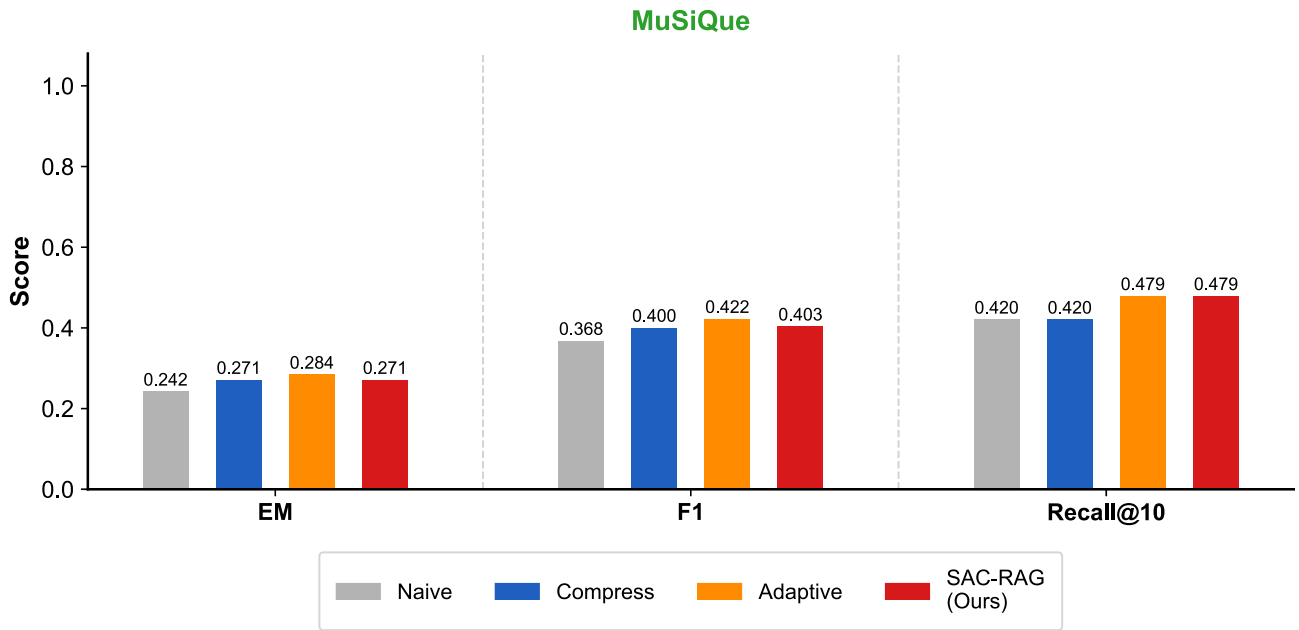


FIGURE 3. Results on MuSiQue comparing four RAG configurations across answer quality (EM, F1) and retrieval recall (Recall@10). MuSiQue features substantially higher retrieval difficulty, serving as a cross-dataset generalization test.

TABLE 4. Configuration and retrieval parameters of the four ablation study groups.

	Naïve (Baseline)	Compress (+Compression)	Adaptive (+Adaptive)	SAC-RAG (Full)
Module Configuration				
Classifier	×	×	✓	✓
Adaptive Retrieval	×	×	✓	✓
Compression	×	✓	×	✓
Retrieval Parameters				
Vector top- <i>k</i>	10→5	10	Strategy-dep.	Strategy-dep.
BM25 (Fact)	–	–	top-10	top-10
BGE Rerank (Fact/Reasoning)	×	×	top-10	top-10
Sub-Q Decomp (Reasoning)	×	×	✓	✓
Final <i>k</i> (Definition)	–	–	8	8

- **H2:** Question-type-adaptive retrieval can improve retrieval precision and answer quality (Adaptive vs Naïve).
- **H3:** Adaptive retrieval and context compression have a positive synergistic effect (SAC-RAG vs Compress; SAC-RAG vs Adaptive).

2) ABLATION RESULTS AND HYPOTHESIS VERIFICATION

H1 validation: Compress RAG outperforms Naïve RAG on all three datasets (EM +1.2/+2.4/+2.9pp), with token reductions of 9–36%. This confirms that directly feeding redundant retrieval results into the generator introduces noise and wastes tokens; context compression effectively reduces both noise and cost.

H2 validation: Adaptive RAG achieves significant EM improvements over Naïve RAG on all datasets

(+5.1/+3.3/+4.2pp), with Recall@10 improvements of 2.5–9.3pp. This confirms that adaptive retrieval is the primary driver of answer quality improvement—selecting a differentiated retrieval strategy based on question type significantly outperforms applying a uniform strategy to all questions.

H3 validation: SAC-RAG further improves EM over Compress RAG (+2.3/+0.4/0.0pp), with Recall@10 gains of 1.9–9.3pp. Better initial retrieval provides higher-quality input for compression. On MuSiQue, SAC-RAG EM is level with Compress but Recall@10 leads substantially (+5.8pp), further confirming the value of adaptive retrieval within the compression pipeline. Taken together, SAC-RAG consistently achieves the optimal quality-efficiency trade-off across all three datasets, trading only 1–2pp of EM for 38–58% token savings relative to Adaptive RAG.

TABLE 5. Ablation experiment results and hypothesis verification.

Hypothesis	Comparison	'EM _{HP}	'EM _{2W}	'EM _{MQ}	'R@10 _{HP}	'R@10 _{2W}	'R@10 _{MQ}
H1: Compression	Compress vs Naïve	+0.012	+0.024	+0.029	+0.001	+0.001	+0.000
H2: Adaptive	Adaptive vs Naïve	+0.051	+0.033	+0.042	+0.093	+0.025	+0.058
H3: Synergy	SAC-RAG vs Compress	+0.023	+0.004	+0.000	+0.093	+0.019	+0.058

To visualize the token consumption patterns more directly, Figure 4 presents the per-query average token consumption of the four configurations across the three datasets. SAC-RAG maintains the lowest token consumption among all configurations on all datasets, while Adaptive RAG—despite achieving the best answer quality—incurs substantially higher token overhead than even the Naïve baseline. Compress RAG alone yields noticeable reductions, confirming that the compression module is the primary driver of token savings.

D. ANALYSIS OF COMPRESSION EFFECTS BY QUESTION TYPE

The ablation results above validate the overall effectiveness of adaptive retrieval and context compression. However, a deeper question constitutes the theoretical premise of SAC-RAG's design—does context compression affect different question types uniformly? If compression effects were type-independent, there would be no need for differentiated retrieval after classification; conversely, if different types respond to compression in significantly different ways, this validates the design rationale of “classify first, then retrieve” in SAC-RAG.

To answer this question, Table 6 presents the per-category performance of Adaptive RAG and SAC-RAG across different question types.

Reasoning questions represent the category where compression is most effective. On HotpotQA, SAC-RAG achieves an EM of 0.577 on Reasoning questions, a 26.8% improvement over Adaptive RAG's 0.455. On 2Wiki, Adaptive RAG's EM on Reasoning questions is as high as 0.859, while SAC-RAG achieves 0.855, essentially on par, with token consumption dropping from 3,255 to 1,490 (−54.2%). For questions involving multi-step reasoning chains, compression effectively filters out noise in the retrieved results, allowing the LLM to focus on critical evidence. On MuSiQue, the Reasoning EM decreases modestly from 0.253 to 0.220 (−3.3 pp), suggesting that when retrieval quality is already low (Recall@10=0.372–0.385), the compressed context may lose critical evidence fragments that are too sparse to survive filtering.

Factual questions constitute the largest category by count. On both datasets, SAC-RAG's EM on Fact questions decreases by 2.4% (HotpotQA: 0.614→0.599) and 4.9% (2Wiki: 0.528→0.502), respectively, compared to Adaptive RAG. Given the 55–60% reduction in token consumption, this trade-off is acceptable in practical applications.

Definition questions have the smallest sample size (HotpotQA 20 instances, 2Wiki 5 instances). These sample

sizes reflect the natural distribution of definitional queries in the original datasets, and the resulting per-category metrics should be interpreted with appropriate caution. On 2Wiki, SAC-RAG EM reaches 1.000 (5/5), outperforming Adaptive RAG's 0.800; on HotpotQA, EM drops from 0.650 to 0.450. A detailed analysis is provided in Section VI.

Synthesizing the per-type results across all three categories, Reasoning questions exhibit the most favorable compression trade-off (EM improves or stays level with large token savings on two of three datasets), Fact questions show slight degradation, and Definition questions are highly sensitive—the response patterns to compression differ markedly by question type. This finding confirms the necessity of type-differentiated retrieval strategies and supports the future design of type-adaptive compression strategies (see Section VI-C).

E. QUALITY-EFFICIENCY PARETO ANALYSIS

Figure 5 illustrates the trade-off between retrieval quality (Recall@10) and token consumption for the four experimental configurations across three datasets. SAC-RAG lies on the Pareto frontier on all datasets, achieving Recall@10 nearly on par with Adaptive RAG at the lowest token consumption. Compress RAG has low token usage but insufficient retrieval quality, while Naïve RAG underperforms on both dimensions.

F. TIME EFFICIENCY

Naïve RAG is the fastest (HotpotQA ~9.6 s/instance), Adaptive RAG adds classification and differentiated retrieval overhead (~21.5 s), and SAC-RAG further adds a compression step (~27.8 s). These additional LLM calls (classifier, sub-question decomposer, compressor) dominate the time overhead. Since the computational cost arises from API calls rather than local GPU computation, GPU memory and FLOPs are not directly comparable; per-query latency is reported as the primary efficiency metric. See Section VI-C for further discussion.

VI. DISCUSSION

A. OVERALL PERFORMANCE OF SAC-RAG

The consistent performance of SAC-RAG across three multi-hop QA datasets validates the effectiveness of combining question-type-adaptive retrieval with context compression. Adaptive RAG achieves the best answer quality on all datasets, with EM scores of 0.611, 0.595, and 0.284 on HotpotQA, 2Wiki, and MuSiQue, respectively. SAC-RAG reduces token consumption by 38–58% at the cost

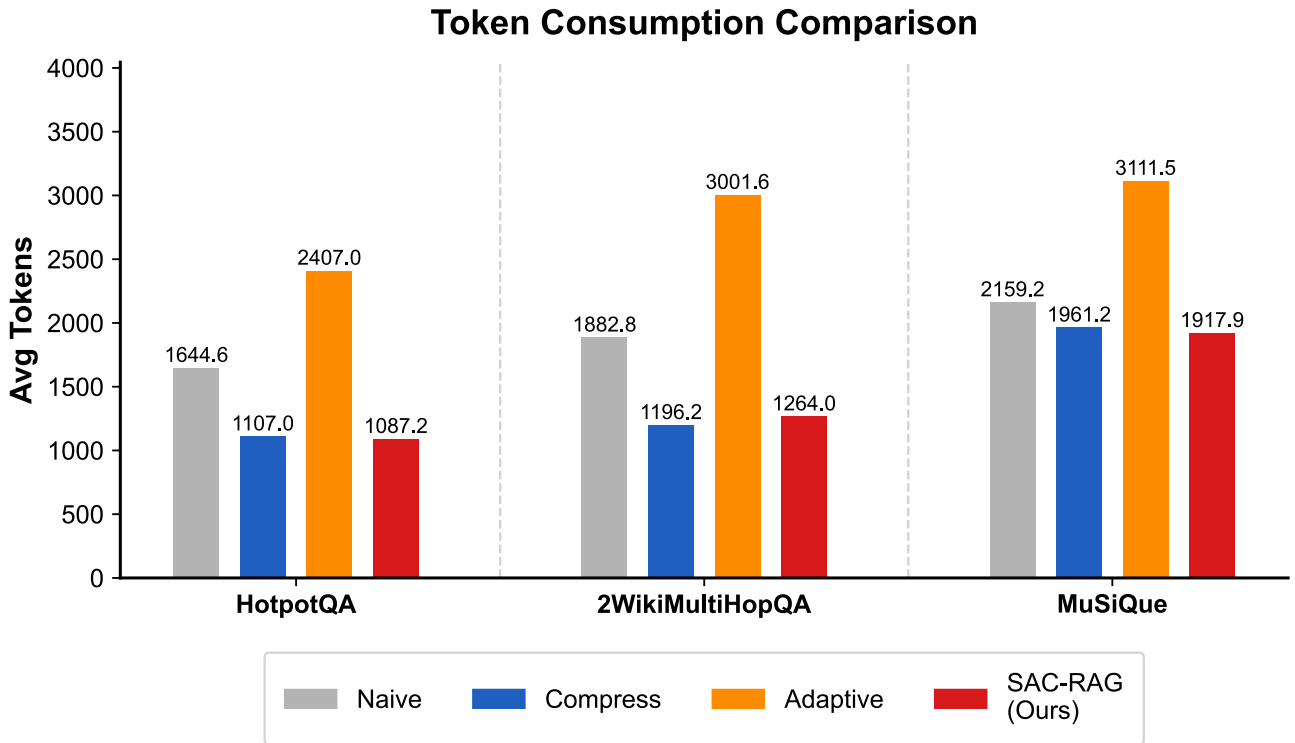


FIGURE 4. Per-query average token consumption across four RAG configurations on three multi-hop QA datasets. SAC-RAG achieves the lowest token consumption on all datasets, with 38%–58% savings relative to Adaptive RAG.

TABLE 6. Per-category results by question type (three datasets).

Dataset	RAG Type	Q-Type	Count	EM	F1	Recall@10	Avg Tokens
HotpotQA	Adaptive RAG	Fact	958	0.614	0.752	0.883	2409
	SAC-RAG	Fact	954	0.599	0.743	0.884	1079
	Adaptive RAG	Definition	20	0.650	0.740	0.775	2143
	SAC-RAG	Definition	20	0.450	0.679	0.800	1232
	Adaptive RAG	Reasoning	22	0.455	0.713	0.841	2565
	SAC-RAG	Reasoning	26	0.577	0.676	0.846	1279
2Wiki	Adaptive RAG	Fact	796	0.528	0.614	0.750	2942
	SAC-RAG	Fact	753	0.502	0.596	0.758	1189
	Adaptive RAG	Definition	5	0.800	0.960	0.900	2339
	SAC-RAG	Definition	5	1.000	1.000	0.800	1553
	Adaptive RAG	Reasoning	199	0.859	0.870	0.536	3255
	SAC-RAG	Reasoning	242	0.855	0.868	0.534	1490
MuSiQue	Adaptive RAG	Fact	724	0.290	0.431	0.521	3056
	SAC-RAG	Fact	739	0.281	0.416	0.514	1775
	Adaptive RAG	Definition	31	0.387	0.468	0.328	3025
	SAC-RAG	Definition	25	0.440	0.518	0.340	2407
	Adaptive RAG	Reasoning	245	0.253	0.389	0.372	3287
	SAC-RAG	Reasoning	236	0.220	0.353	0.385	2313

of only 1–2 percentage point EM drops, demonstrating the complementarity of the two components. On the most challenging MuSiQue dataset, SAC-RAG’s Recall@10

remains exactly level with Adaptive RAG (0.479), further validating the framework’s reliability across different retrieval difficulties.

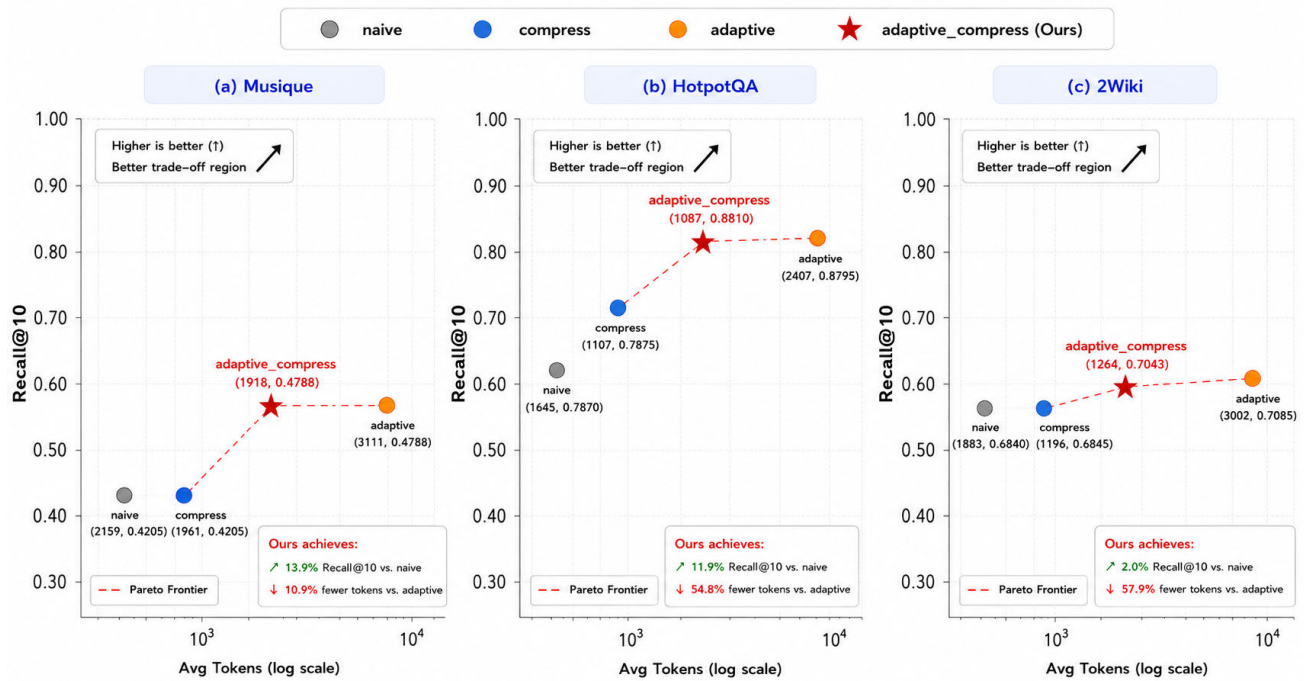


FIGURE 5. Pareto frontier of retrieval quality (Recall@10, y-axis) against token consumption (x-axis) across three datasets. The upper-left region is optimal (high recall, low tokens); the lower-right is inferior. Each marker denotes one RAG configuration; solid lines trace Pareto-optimal points. SAC-RAG lies on the frontier on all three datasets, reaching near-optimal recall at the lowest token consumption.

TABLE 7. HotpotQA compression degradation analysis (F1 basis).

Question Type	Total	Degraded	Ratio	Avg F1 Drop
Fact	958	63	6.6%	0.568
Definition	20	2	10.0%	0.600
Reasoning	22	1	4.5%	0.738

B. ERROR ANALYSIS OF COMPRESSION DEGRADATION

To understand under what circumstances context compression leads to degraded answer quality, we conduct a fine-grained degradation analysis on the HotpotQA dataset. Comparing the per-question F1 scores of Adaptive RAG and SAC-RAG, the results are shown in Table 7.

Definition-type questions are the most sensitive to compression. By comparing the retrieved documents before and after compression for the same question, we find that the top-8 retrieved documents are identical, ruling out retrieval differences as the cause of degradation and confirming that the issue lies in the post-compression answer generation stage. A representative case:

Question: “Who is this American cartoonist, writer, producer, animator, and voice actor that worked with this multiple Shuster Award, Harvey Award and Eisner Award nominee and an Eisner Award-winning comic book creator?”

Ground truth: Matt Groening

Adaptive RAG: Matt Groening ✓

SAC-RAG: Steve Purcell ✗

The retrieval context simultaneously contains relevant documents about both Matt Groening and Steve Purcell. In the full context, the LLM correctly identifies the target person; however, in the compressed and refined context, the distinguishing cues between the two entities are weakened. This case reveals the core risk of compression for definition-type questions: when multiple entities of the same type co-occur in the context, compression may weaken the distinguishing cues between them.

Fact-type questions show a slight but systematic drop. Among 958 Fact questions, 63 (6.6%) show F1 drops. The typical degradation pattern is the loss of key factual details during compression. For instance, the question “Where was the place of death of Vigrahapala I (Pala Dynasty)’s father?” has the ground-truth answer Peshawar; Adaptive RAG answers correctly, while SAC-RAG incorrectly answers Bengal.

Reasoning-type questions benefit from compression. Among the 22 Reasoning questions in HotpotQA, only 1 shows degradation, and SAC-RAG’s EM on the Reasoning category improves from 0.455 to 0.577 (+26.8%). Compression filters out noisy documents in the reasoning path, allowing the LLM to focus on core evidence fragments.

C. LIMITATIONS

(1) **Limited classifier granularity.** The current three-type classification system covers the main QA categories, but real-world user queries may exhibit more diverse semantic categories (e.g., comparison, process, opinion).

- (2) **Imbalanced subtype samples.** The Definition class contains only 20 samples in HotpotQA (2.0%) and 5 in 2Wiki (0.5%); the Reasoning class has only 22 samples in HotpotQA (2.2%). Conclusions on these subtypes have limited statistical stability due to insufficient sample sizes.
- (3) **Model dependence.** The answer generation and context compression in this study rely on gpt-5-mini and qwen-flash, respectively; different LLMs may exhibit different generation capabilities and compression behaviors. However, internal validity is ensured through controlled variable comparisons under the same LLM configuration.
- (4) **Single-run evaluation.** All experiments were conducted under deterministic decoding (temperature = 0) to minimize generation variance. Future work should include repeated trials to further quantify pipeline-level variance introduced by the retriever, classifier, and compression modules.
- (5) **Dataset coverage.** Current experiments are conducted on three English multi-hop QA datasets. Generalization performance on non-English datasets and vertical domain datasets remains to be verified.
- (6) **Classifier error propagation.** Despite the satisfactory standalone performance of the classifier (92.0% accuracy), misclassification may still propagate to downstream retrieval strategy selection, compression, and generation stages. The impact of classifier errors on end-to-end performance warrants further investigation.

D. FUTURE WORK

- (1) **Type-adaptive compression strategies.** Given that definition-type questions are the most sensitive to compression, future work can explore dynamically adjusting compression intensity for different question types.
- (2) **Classification system expansion.** The classification system can be expanded to a finer-grained taxonomy, or a lightweight classifier without LLM invocation can be introduced to reduce latency and token overhead.
- (3) **Cross-domain validation.** Validate the generalization capability of the SAC-RAG framework on non-English corpora and specific vertical domains.

VII. CONCLUSION

This paper proposes SAC-RAG, a unified framework that integrates question-type-adaptive retrieval with context compression. SAC-RAG uses an LLM classifier to categorize queries in real time into factual, definitional, and reasoning types, and employs differentiated retrieval strategies for each type—Fact-type questions use hybrid BM25 and dense vector retrieval with reranking, Definition-type questions adopt pure semantic retrieval, and Reasoning-type questions undergo sub-question decomposition followed by parallel retrieval—after which all retrieval results pass through a unified LLM context compression module to extract evidence snippets before generating the final answer.

Results from four progressive ablation experiments on HotpotQA, 2WikiMultiHopQA, and MuSiQue demonstrate that: (1) adaptive retrieval is the primary driver of answer quality improvement, boosting EM by 3.3–5.1 percentage points and Recall@10 by 2.5–9.3 percentage points; (2) context compression reduces token consumption by 38–58% with EM decreasing by only 1–2 percentage points; (3) SAC-RAG achieves the optimal quality–efficiency trade-off, lying on the Pareto frontier on all three datasets; for reasoning-type questions, compression improves answer quality by filtering out noise.

The time overhead (approximately 27.8 s vs. 9.6 s per instance for Naïve RAG on HotpotQA) primarily stems from the additional LLM calls for classification, sub-question decomposition, and compression. This trade-off between token efficiency and latency should be weighed against application-specific deployability requirements. The success of SAC-RAG confirms that optimizing retrieval by question type before applying uniform compression provides a practical solution for reducing LLM API costs in real-world deployments while maintaining answer quality.

REFERENCES

- [1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, Feb. 2024.
- [2] R. Zhu, X. Liu, Z. Sun, Y. Wang, and W. Hu, “Mitigating lost-in-retrieval problems in retrieval augmented multi-hop question answering,” in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (Long Papers)*, 2025, pp. 22362–22375.
- [3] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol. (Long Papers)*, 2024, pp. 7036–7050.
- [4] J. Fang, Z. Meng, and C. MacDonald, “KiRAG: Knowledge-driven iterative retriever for enhancing retrieval-augmented generation,” in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (Long Papers)*, 2025, pp. 18969–18985.
- [5] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 13358–13376.
- [6] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 2369–2380.
- [7] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, “Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps,” in *Proc. 28th Int. Conf. Comput. Linguistics*, 2020, pp. 6609–6625.
- [8] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “MuSiQue: Multihop questions via single-hop question composition,” *Trans. Assoc. Comput. Linguistics*, vol. 10, pp. 539–554, May 2022.
- [9] P. Lewis, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.
- [10] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6769–6781.
- [11] Y. Gao, “Retrieval-augmented generation for large language models: A survey,” *Tech. Rep.*, 2024.
- [12] A. Brown, M. Roman, and B. Devereux, “A systematic literature review of retrieval-augmented generation: Techniques, metrics, and challenges,” *Big Data Cognit. Comput.*, vol. 9, no. 12, p. 320, Dec. 2025.

- [13] G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave, "Atlas: Few-shot learning with retrieval augmented language models," *J. Mach. Learn. Res. (JMLR)*, vol. 24, no. 251, pp. 1–43, 2023.
- [14] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, and Y. Shoham, "In-context retrieval-augmented language models," *Trans. Assoc. Comput. Linguistics*, vol. 11, pp. 1316–1331, Nov. 2023.
- [15] N. Thakur, N. Reimers, A. Rüklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. 35th Conf. Neural Inf. Process. Syst. Datasets Benchmarks Track*, 2021.
- [16] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024.
- [17] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 7969–7992.
- [18] M. R. Rezaei and A. B. Dieng, "Vendi-RAG: Adaptively trading-off diversity and quality significantly improves retrieval augmented generation with LLMs," Tech. Rep., 2025.
- [19] W. Shi, S. Min, M. Yasunaga, M. Seo, R. James, M. Lewis, L. Zettlemoyer, and W.-T. Yih, "REPLUG: Retrieval-augmented black-box language models," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 1–19, Jun. 2024.
- [20] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting in retrieval-augmented large language models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 5303–5315.
- [21] O. Young, Y. Fan, R. Zhang, J. Guo, M. de Rijke, and X. Cheng, "GaQR: An efficient generation-augmented question rewriter," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2024, pp. 4228–4232.
- [22] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 1–11.
- [23] J. Zhang, X. Liu, Y. Hu, C. Niu, F. Wu, and G. Chen, "RAGRouter: Learning to route queries to multiple retrieval-augmented language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2025.
- [24] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Long Papers)*, 2024, pp. 1658–1677.
- [25] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024.
- [26] Z. Li, L. Peng, Y. Zhang, and J. Zhang, "BRIEF: Bridging retrieval and inference for multi-hop reasoning via compression," in *Proc. Findings Assoc. Comput. Linguistics: NAACL*, 2025.
- [27] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, "LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," in *Proc. Findings Assoc. Comput. Linguistics ACL*, 2024, pp. 963–981.
- [28] Y. Jin, K. Sharma, V. Rakesh, Y. Dou, M. Pan, M. Das, and S. Kumar, "SARA: Selective and adaptive retrieval-augmented generation with context compression," in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (Long Papers)*, 2026, pp. 14508–14528.
- [29] C. Zhang, Y. Wang, D. Xu, H. Zhang, Y. Lyu, Y. Chen, S. Liu, T. Xu, X. Zhao, Y. Gao, Y. Hu, and E. Chen, "TeaRAG: A token-efficient agentic retrieval-augmented generation framework," 2025, *arXiv:2511.05385*.
- [30] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 24824–24837.
- [31] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [32] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2023, pp. 10014–10037.
- [33] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, "Decomposed prompting: A modular approach for solving complex tasks," in *Proc. 11th Int. Conf. Learn. Represent.*, 2023.
- [34] O. Khattab, K. Santhanam, X. L. Li, D. Hall, P. Liang, C. Potts, and M. Zaharia, "Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive NLP," Tech. Rep., 2022.
- [35] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-pack: Packaged resources to advance general Chinese embedding," Tech. Rep., 2023.
- [36] W. Feng, C. Hao, Y. Zhang, G. Jiang, and J. Song, "AirRAG: Autonomous strategic planning and reasoning steer retrieval augmented generation," in *Proc. Findings Assoc. Comput. Linguistics EMNLP*, 2025, pp. 18934–18953.



DEYU ZHANG received the Ph.D. degree in computer technology from Nanjing University of Science and Technology, Nanjing, China, in 2005. He is currently a Professor with the School of Information Science and Engineering, Shenyang Ligong University, Shenyang, Liaoning, China.

His research interests include target recognition, artificial intelligence, computer vision, and mobile ad hoc networks.



HONGQIANG YU is currently pursuing the M.S. degree in computer technology with the School of Information Science and Engineering, Shenyang Ligong University, Shenyang, Liaoning, China.

His research interests include big data and intelligent information processing and artificial intelligence applications.



JINZE HUO received the B.S. degree in digital media technology from Northeastern University, Shenyang, China, in 2018, and the M.S. and Ph.D. degrees in computer science from Loughborough University, Loughborough, U.K., in 2019 and 2025, respectively. He is currently a Lecturer with the School of Information Science and Engineering, Shenyang Ligong University, Shenyang, Liaoning, China.

His research interests include action recognition, robot controlling, target tracking, artificial intelligence, deep learning, computer vision, and machine learning.



MO SU received the Ph.D. degree in computer applied technology from the University of Chinese Academy of Sciences, Beijing, China, in 2019. He is currently a Professor with the School of Information Science and Engineering, Shenyang Ligong University, Shenyang, Liaoning, China.

His research interests include artificial intelligence, virtual reality and visualization, and data engineering.

...